Ethan Patterson

HW1- Problem 4

a.) The algorithm is a solution to the smallest value-most sums problem. It finds every possible sum given a series of numbers, and returns the value that appears the most times as a sum. In the case of a tie for occurrences, the smaller value is taken.

b.) pseudo code:

```
Numbers[] = [input go here]

sumHeap = new double[numbers.size * numbers.size]
For i = 0; i < numbers.size; i++
        For j=i+1; j<numbers.size; j++
                Add_to_heap(sumheap, numbers[i]+numbers[j])

Bestcount = 1
bestValue = extractMax(sumHeap)

Currentcount = 1
previousValue = bestValue

for(i=sumheap.size; i> 0; i–)
        Currentvalue = extractMax(sumheap)
        If currentvalue = previousValue: currentcount++
        Else
                if(currentcount >= bestCount
                        bestCount = currentCount
                        bestValue = previousValue
                currentCount = 1
                previousValue = currentvalue

Print bestCount and bestValue
```

c.) Tossing the sums in a max heap will sort them, biggest first. Once we calculate every possible sum, we can then count the number of occurrences of a value by seeing if the previous value and the current value are the same simply by just pulling each max out of the heap until it's empty. From here, it's easy enough to derive the values needed for the output simply by storing the current best value and best count, then updating it as needed when a larger count/smaller value with the same count is found.

d.) $O(n^2 \log(n))$

e.) The code boils down to doing n^2 insertions to a heap which is logn per, and then another n^2 retrievals which is also logn per. So the result is 2(n^2 log(n)) which collapses down to just n^2 log(n)